Fast algorithm for terrain irradiance using compressed view factor matrices

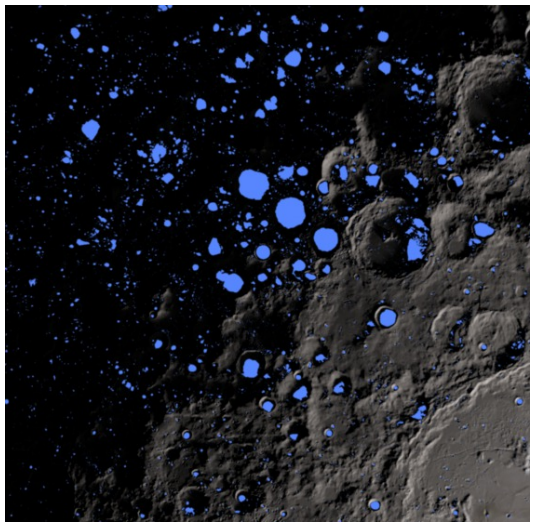
Samuel F. Potter

Courant Institute of Mathematical Sciences

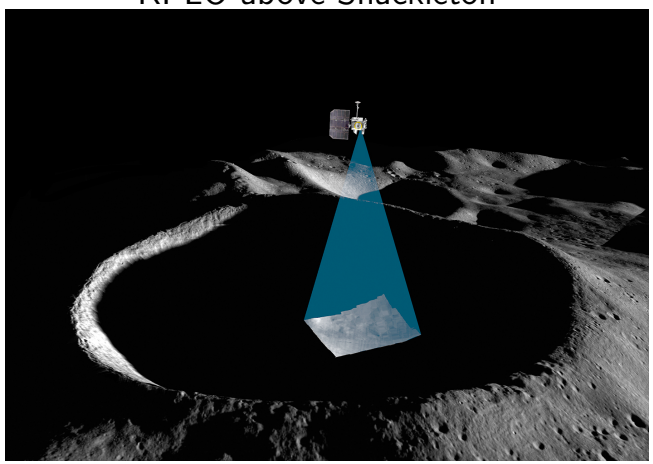
Joint work with Stefano Bertone (NASA), Norbert Schörghofer (PSI), and Erwan Mazarico (NASA).

Joint work with Erwan Mazarico (NASA), Norbert Schörghofer (PSI), and Stefano Bertone (NASA).

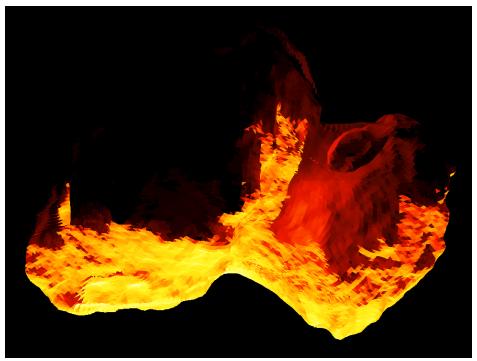
NASA SVS: "LRO Peers Into Permanent Shadows"[†]



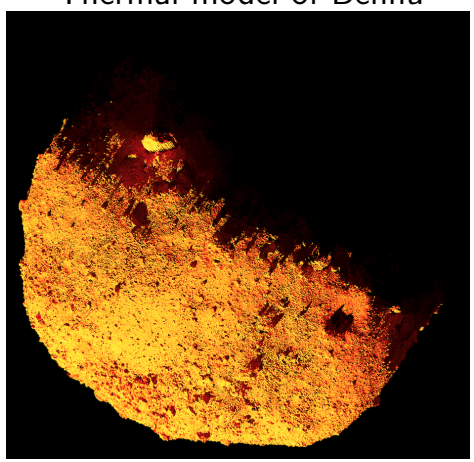
NASA ShadowCam on KPLO above Shackleton[§]



Thermal model of 67P



Thermal model of Bennu



- **Goal**: develop fast algorithms for Lambertian scattering
- Solve long-running thermal simulations (forward problem)
- Ultimate application is in inverse problems
- Target application is the study of Lunar PSRs

The NASA LROC imaged Lunar PSRs at approx. 20 m/px.
ShadowCam will image PSRs at approx. 1.7 m/px.

Problem setup

Let S be a rough planetary surface (a large crater, small asteroid, etc.).

Energy balance on S is:

$$(1-\alpha)(Q_{\text{direct}} + Q_{\text{refl}}) + k\hat{n} \cdot \nabla T + \epsilon Q_{\text{IR}} = \epsilon\sigma T^4$$

where α is albedo, Q_{direct} is insolation, Q_{refl} is short wavelength terrain irradiance, Q_{IR} is long wavelength terrain irradiance, k is thermal conductivity, T is temperature, $\hat{n}$ is surface normal, ϵ is emissivity, and σ is the Stefan-Boltzmann constant.

Without subsurface heat conduction, energy balance is:

$$(1-\alpha)(Q_{\text{direct}} + Q_{\text{refl}}) + \epsilon Q_{\text{IR}} = \epsilon\sigma T_{\text{eq}}^4.$$

Problem setup

Insolation depends on position of the sun:

$$Q_{\text{direct}} = \frac{S_\odot}{R_\odot^2} \sin(e_\odot) V(\boldsymbol{x}, \boldsymbol{x}_\odot),$$

where

$$V(\boldsymbol{x}, \boldsymbol{y}) = \begin{cases} 1 & \text{if } \boldsymbol{y} \text{ is line-of-sight visible from } \boldsymbol{x}, \\ 0 & \text{otherwise.} \end{cases}$$

is the visibility kernel.

Note: can find sun position using SPICE* and can evaluate V using a raytracer (we use CGAL†).

*https://naif.jpl.nasa.gov/naif/toolkit.html

†https://www.cgal.org/

Problem setup

Terrain irradiances satisfy the two integral equations

$$Q_{\text{refl}}(\boldsymbol{x}) = \int_S F(\boldsymbol{x}, \boldsymbol{y})\alpha(\boldsymbol{y})\Big[Q_{\text{direct}}(\boldsymbol{y}) + Q_{\text{refl}}(\boldsymbol{y})\Big]\, dA(\boldsymbol{y}),$$

$$Q_{\text{IR}}(\boldsymbol{x}) = \int_S F(\boldsymbol{x}, \boldsymbol{y})\Big[\epsilon(\boldsymbol{y})\sigma\, T(\boldsymbol{y})^4 + (1 - \epsilon(\boldsymbol{y}))\, Q_{\text{IR}}(\boldsymbol{y})\Big]\, dA(\boldsymbol{y}),$$

where F is the *view factor*:

$$F(\boldsymbol{x}, \boldsymbol{y}) = \frac{\cos\big(\theta(\boldsymbol{x}, \boldsymbol{y})\big)\cos\big(\theta(\boldsymbol{y}, \boldsymbol{x})\big)}{\pi|\boldsymbol{x} - \boldsymbol{y}|^2} V(\boldsymbol{x}, \boldsymbol{y}),$$

and where $\theta(\boldsymbol{x}, \boldsymbol{y})$ is the angle between the unit surface normal at $\boldsymbol{x}$ and the unit vector pointing from $\boldsymbol{x}$ to $\boldsymbol{y}$.

Both of these integral equations are "radiosity" integral equations.

Problem setup

Discretize each integral equation using constant basis functions and point collocation (very simple).

Get the *radiosity kernel matrix*:

$$\boldsymbol{F}_{i,j} = F(x_i, x_j)A_j = \frac{\Big[\hat{n}_i \cdot (x_j - x_i)\Big]\Big[\hat{n}_j \cdot (x_i - x_j)\Big]}{\pi |x_i - x_j|^4} V(x_i, x_j)A_j.$$

Linear system for discretized Q_{refl} is:

$$\boldsymbol{Q}_{\text{refl}} = \boldsymbol{F}\,\text{diag}(\boldsymbol{\alpha})\Big[\boldsymbol{Q}_{\text{direct}} + \boldsymbol{Q}_{\text{refl}}\Big],$$

and similarly for Q_{IR}.

These are well-conditioned systems (discretized second-kind IEs). Easy to solve in $O(1)$ iterations.

The challenge

The $\boldsymbol{F}$ matrix has many zeros, but still has $O(N^2)$ nonzeros overall, hence slow $O(N^2)$ MVP.

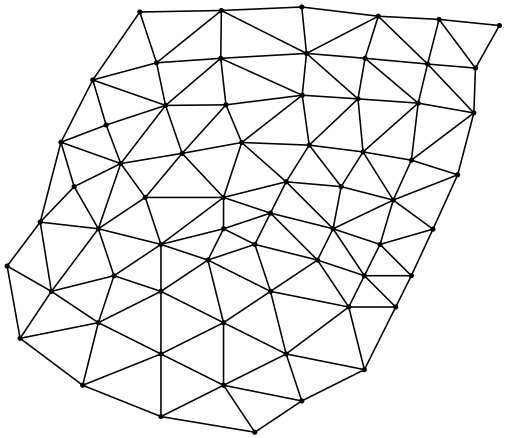
Existing fast algorithms for radiosity from computer graphics fail for rough surfaces discretized into many small triangles.

Instead: hierarchically compress $\boldsymbol{F}$ using spatial data structures and low-rank compression (e.g., SVD), leading to the *compressed view factor matrix*.

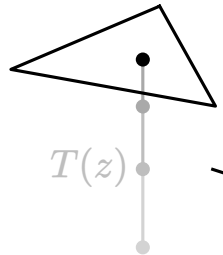
Goal: $\boldsymbol{F}$ only depends on the surface S, so accept an $O(N^2)$ setup cost in order to get $\sim O(N)$ MVP. Amortize setup cost over **many** RHSs.

Details of our approach in paper which will be submitted soon.

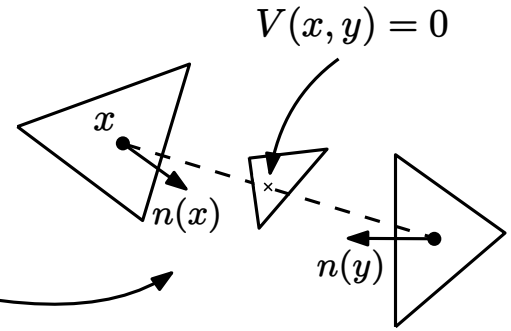
Discretize surface into large tri mesh (easily 3-10M tris)



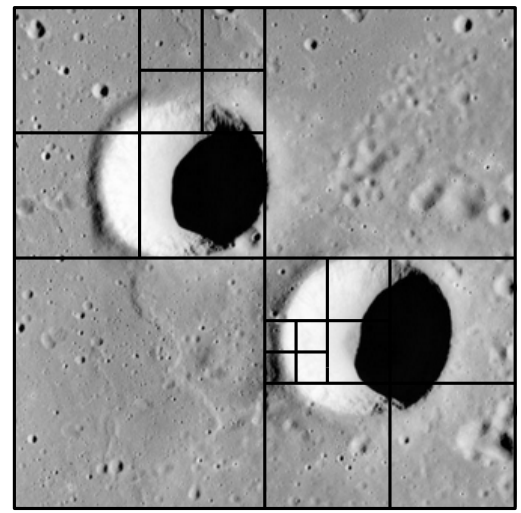
Place 1D thermal model at centroid of each tri



Discretize view factors using midpoint rule



Recursively partition triangles using spatial data structure (quadtree, octree, . . .)



assembly

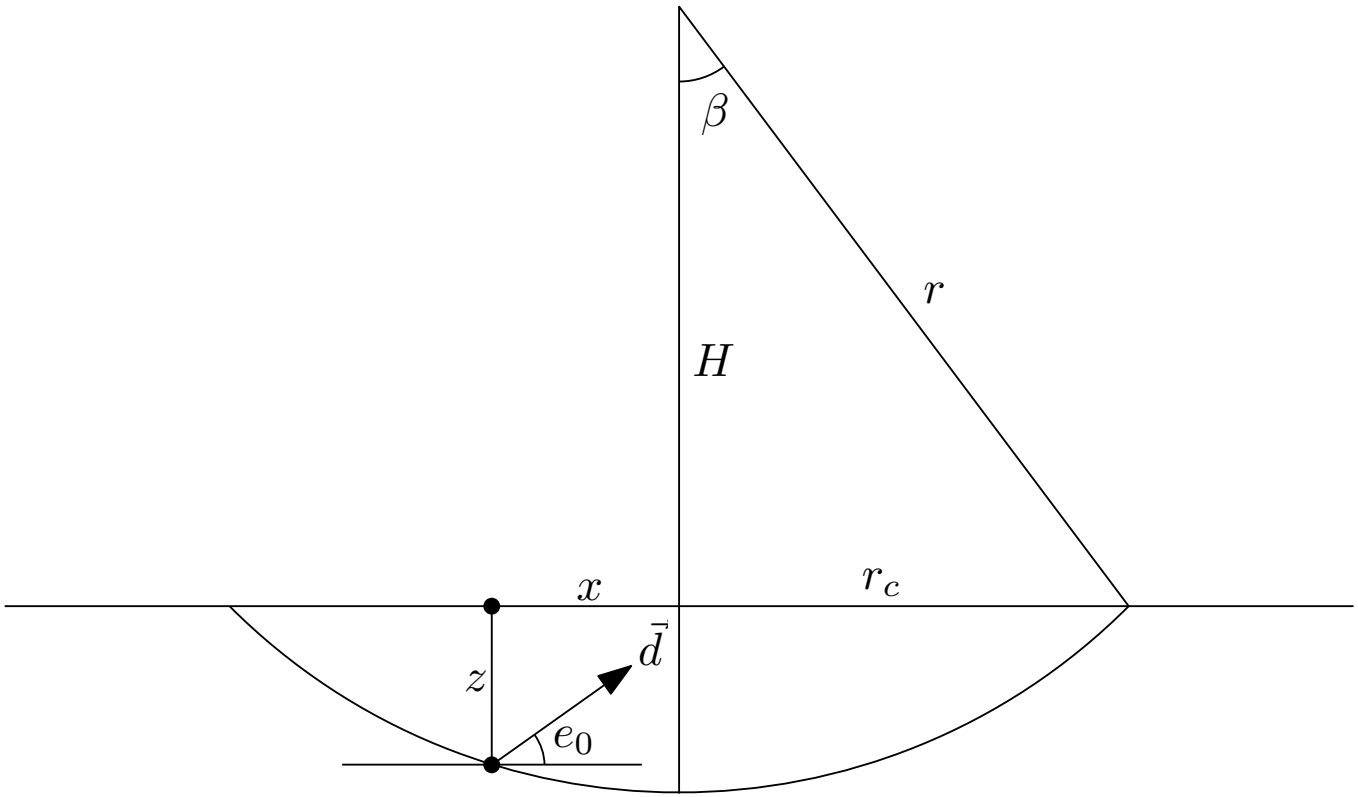
Compress off-diagonal blocks using *either* truncated SVD *or* sparse matrix, depending on which uses less space

Solve system using simple iterative method (e.g., Jacobi converges in 3–8 iterations)

Drive 1D heat models at each triangle, re-radiated thermal emission for next time step

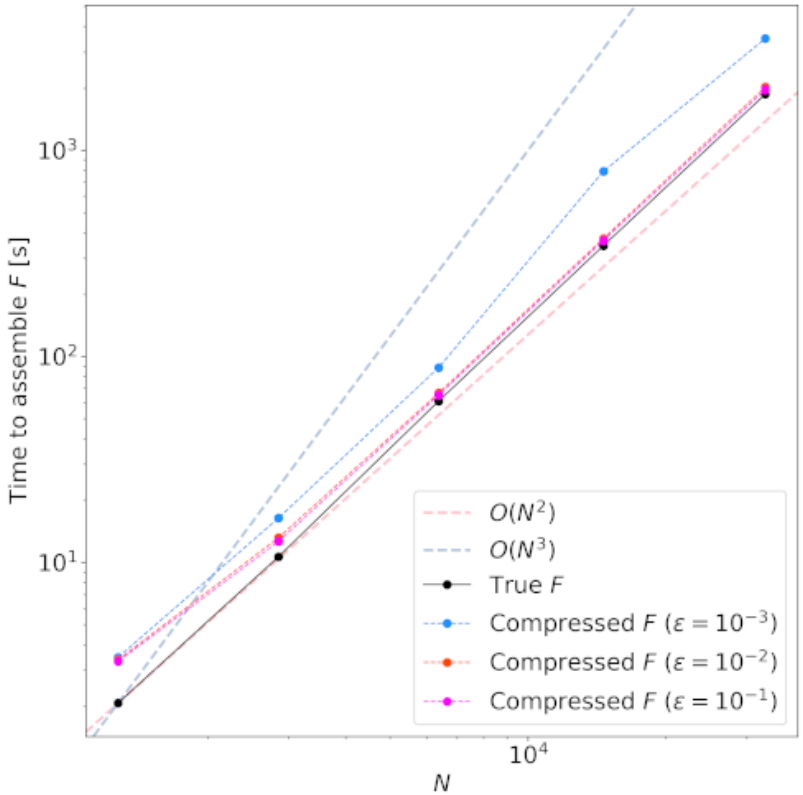
thermal simulation

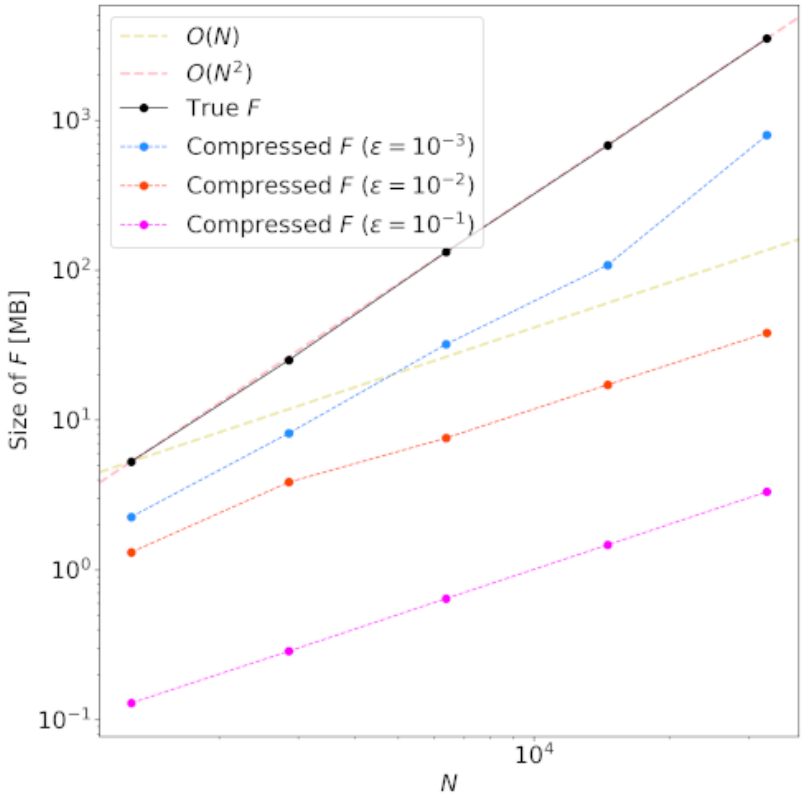
Test problem: spherical cap crater



For a hemispherical crater, the temperature in the shadow zone is constant and can be computed analytically. We use this temperature as a groundtruth for our method.

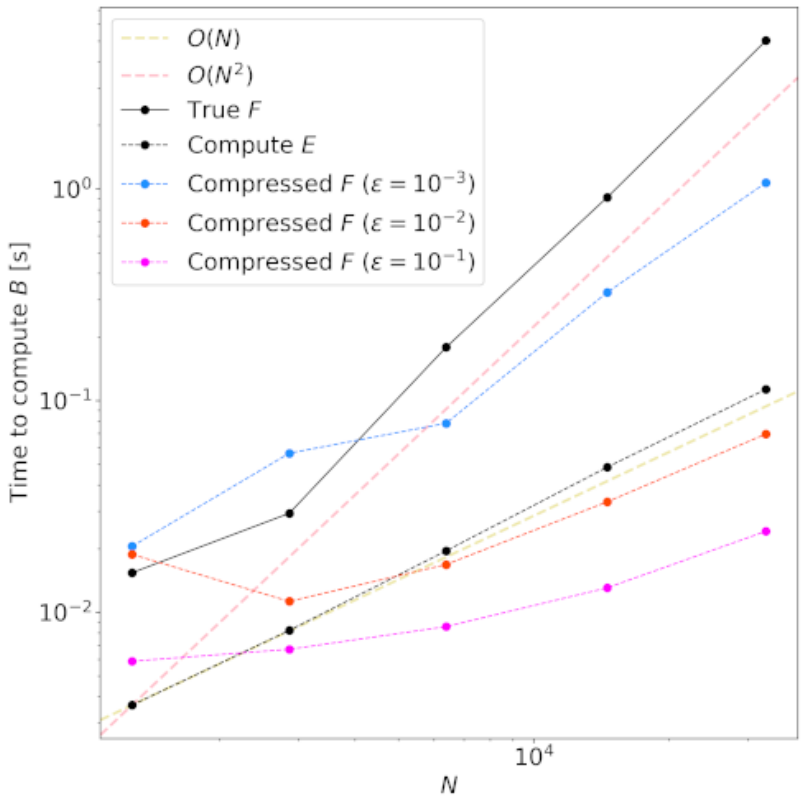
Test problem: spherical cap crater

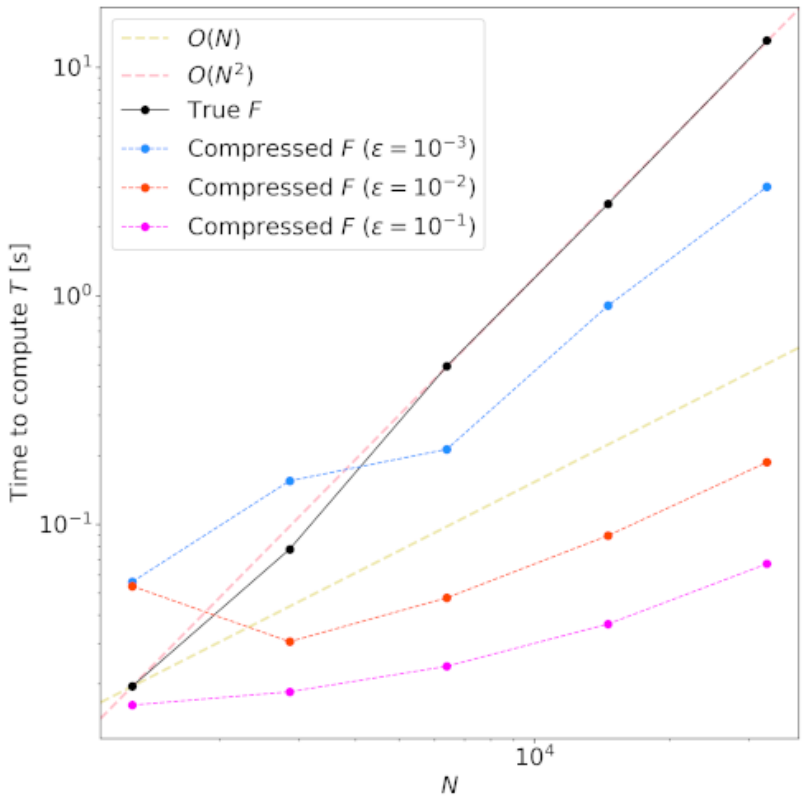




Assembling the compressed view factor matrix takes roughly the same time as computing $\boldsymbol{F}$ itself. The compressed size generally scales as $O(N)$ for coarse enough tolerance.

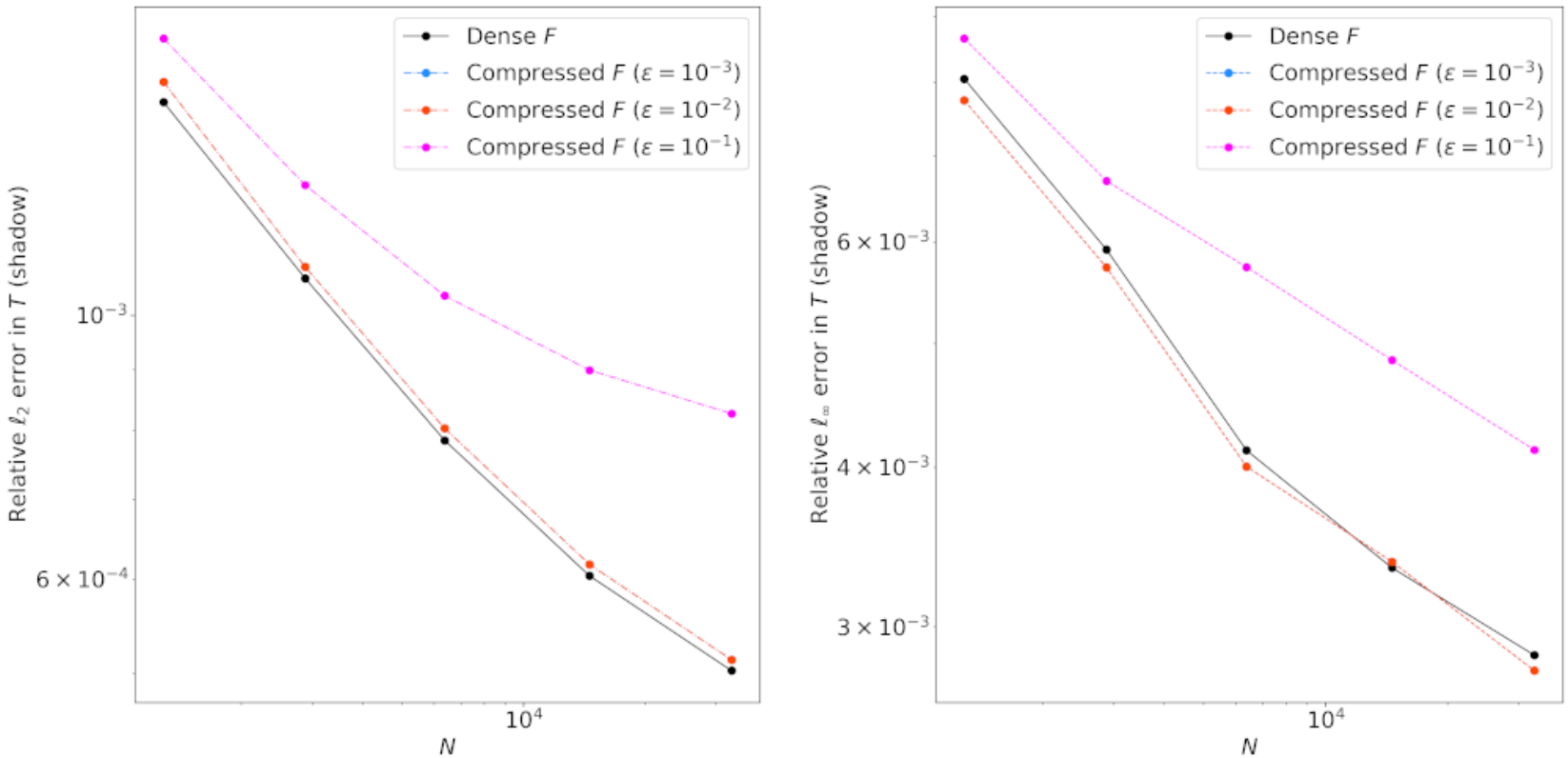
Test problem: spherical cap crater





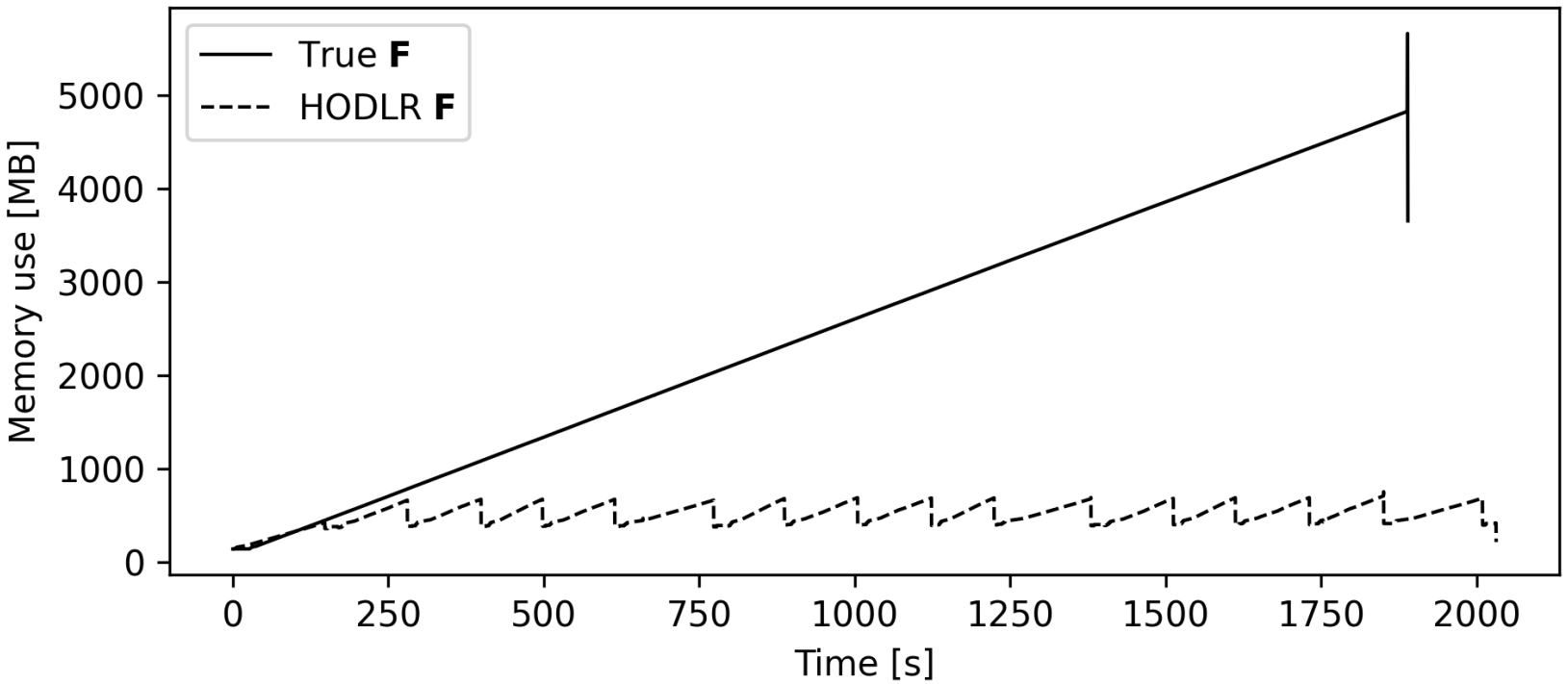
Computing the radiance and temperature both take roughly $O(N)$ time (with a small constant) for coarse enough tolerance.

Test problem: spherical cap crater



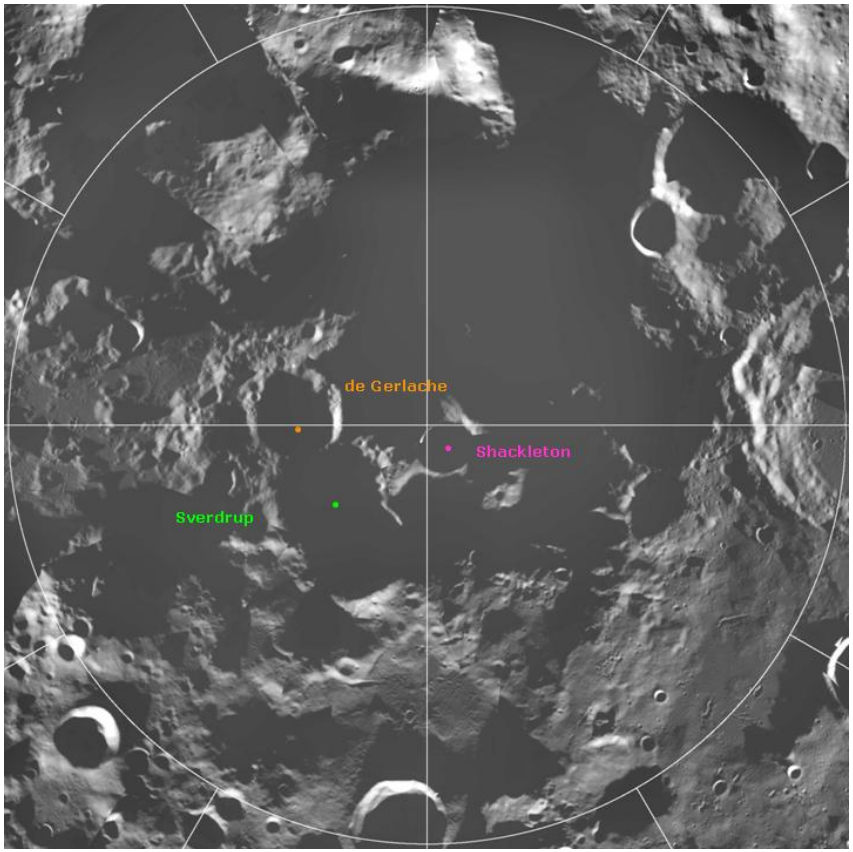
For *small* enough tolerance, relative error is as good as uncompressed $\boldsymbol{F}$ for spherical crater. Happens quickly because of large discretization error in computed solution.

Memory use during assembly



Assembly formally requires $O(N^2)$ space to run, but it has a much smaller constant than assembling $\boldsymbol{F}$ directly. It is necessary necessary to construct all of $\boldsymbol{F}$ at once—we just need individual subblocks.

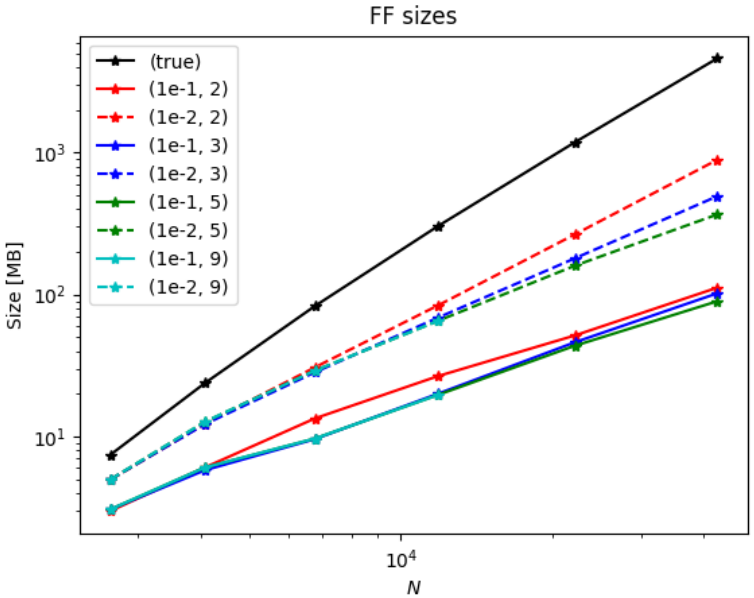
Example: de Gerlache crater (Lunar south pole)

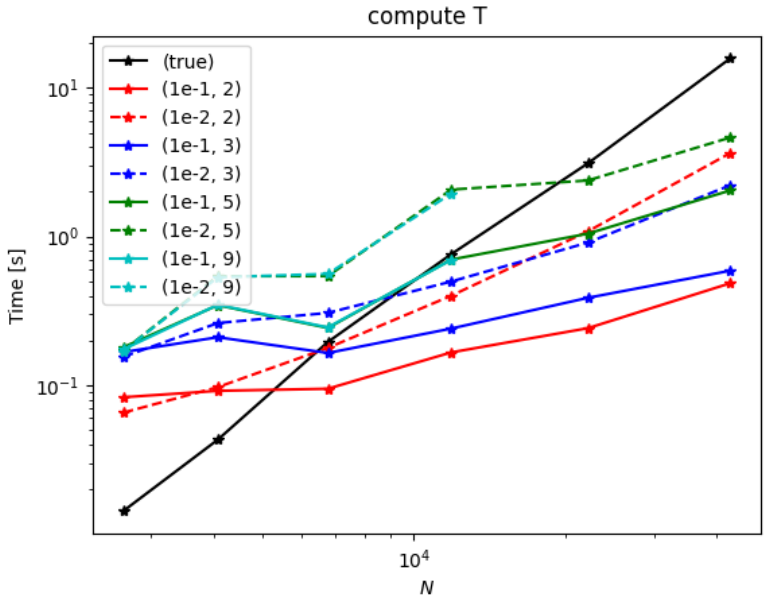


Use a recent, high-resolution digital elevation model (DEM) of Lunar south pole[‡] to do simulation of de Gerlache crater, which is a large PSR.

[‡] Barker et al., 2021.

Example: de Gerlache crater (Lunar south pole)

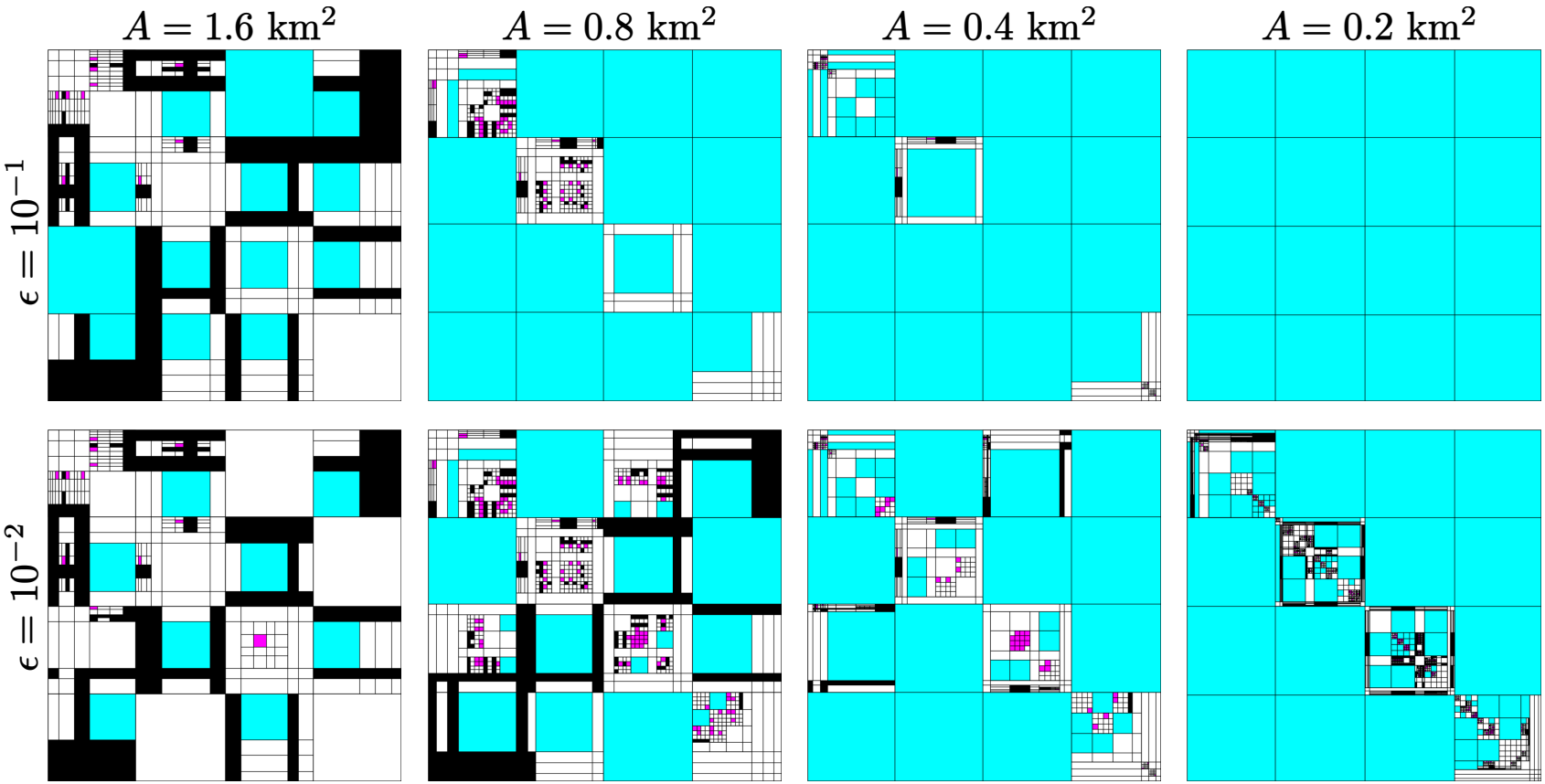




Problem is significantly harder than spherical cap crater because of complicated visibility. Still, for right choice of parameters get $O(N)$ scaling in time and space.

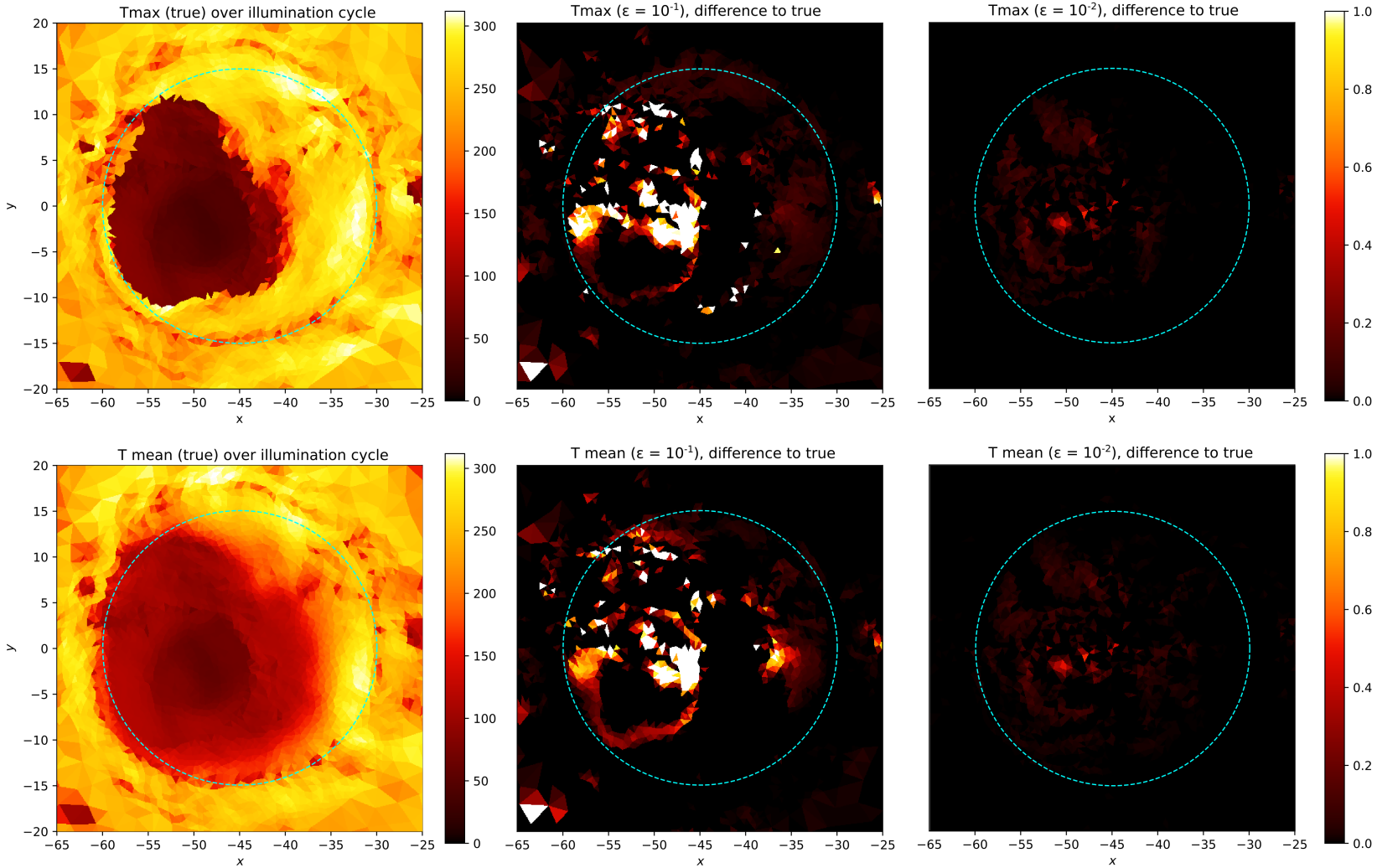
Note: relatively large constant is due to inefficient Python prototype.

Example: de Gerlache crater (Lunar south pole)



Schematic of hierarchically compressed view factor matrices for different triangle mesh finenesses. (**Cyan**: SVD block, **black**: zero block, **white**: sparse CSR block, **magenta**: dense block.)

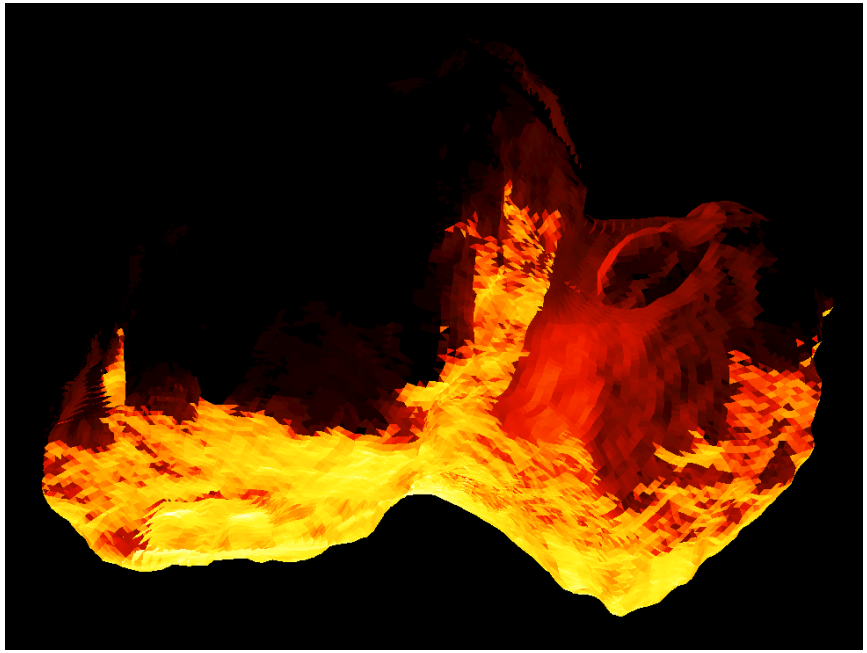
Example: de Gerlache crater (Lunar south pole)



Compare maximum and mean temperatures over an illumination cycle computed using $\boldsymbol{F}$ and its compressed version for two different tolerances.

Example: 67P/Churyumov-Gerasimenko

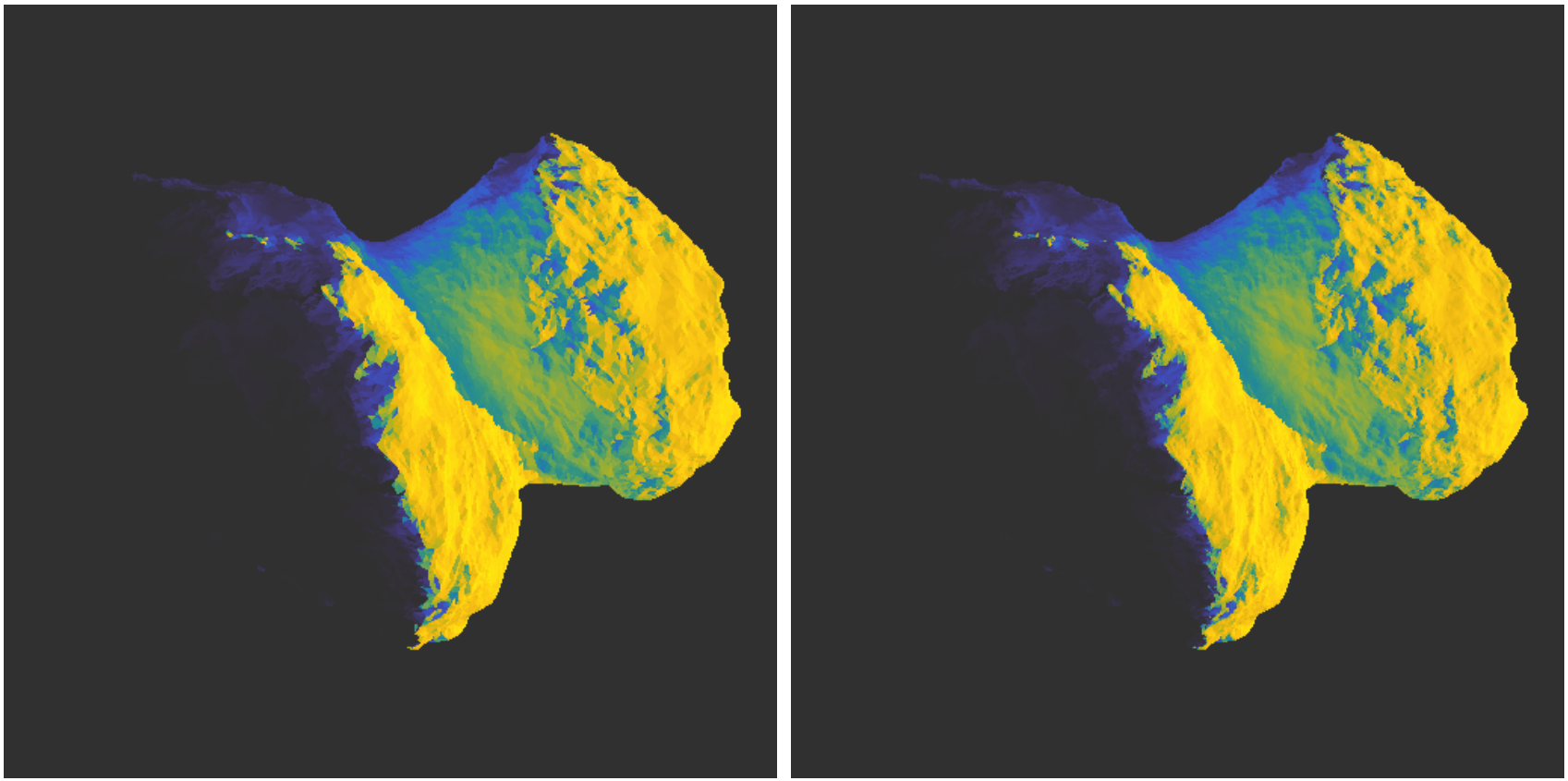




Another example: use an octree do discretize a shape model of 67P. Number of nonzeros in $\boldsymbol{F}$ is much smaller than de Gerlache crater. Nevertheless, our method usefully compresses $\boldsymbol{F}$.

Example: 67P/Churyumov-Gerasimenko

Steady state temperature distribution computed using compressed view factor matrix for different sized meshes:

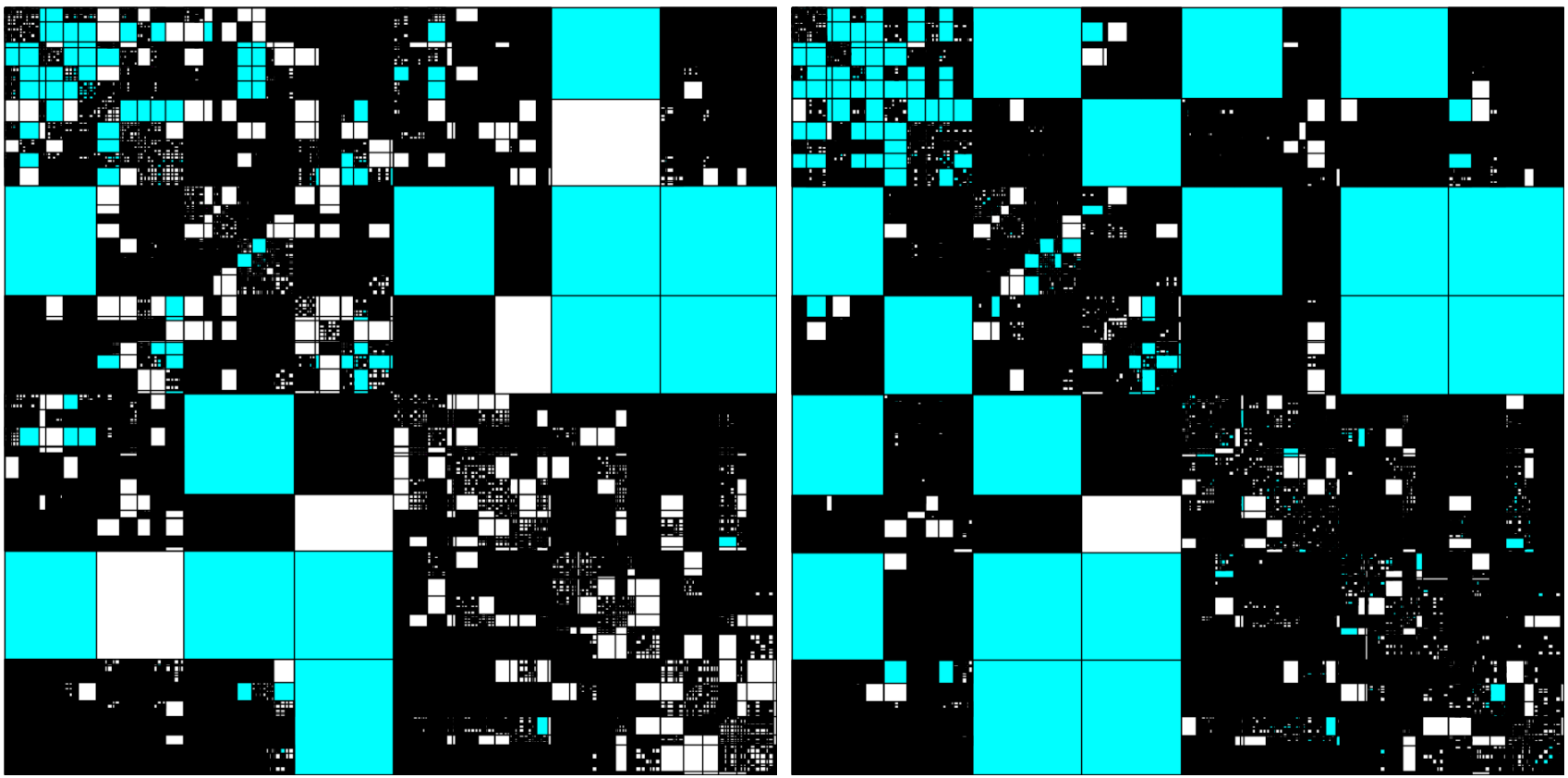


(a) ~49K faces

(b) ~197K faces

Example: 67P/Churyumov-Gerasimenko

Hierarchical view factor matrix blocks (**Cyan**: sparse SVD block, **black**: zero block, **white**: CSR block):



(a) ~49K faces

(b) ~197K faces

Summary

Presented a new algorithm for hierarchically compressing the view factor matrix, suitable for use in modeling temperature irradiance on rough planetary surfaces.

Presented results empirically demonstrating improved asymptotic scaling.

Paper detailing method will be submitted soon.

Prototype implementation available:

`https://github.com/sampotter/python-flux`

Future work: optimize implementation (rewrite in C or C++), accelerate assembly by taking more advantage of available visibility information.